

ДОМАШНЕЕ ЗАДАНИЕ №2 · ФИНТЕХ

Платёжный мини-API с идемпотентностью

Вариант C1 · собрано несколькими агентами по спецификации

Автор: Elnar Aksaitov · Формат: соло

Дата: 29 августа 2026 · Стек: Go (стандартная библиотека)

Оглавление

Резюме (полстраницы)

Обязательная часть

Необязательная часть

Что не сработало / трение (честно)

Ссылки на артефакты

Отчёт по ДЗ №2

Вариант: C1 — платёжный мини-API с идемпотентностью. **Формат:** соло. **Дата:** 29 августа 2026. **Стек:** Go (стандартная библиотека), без внешних зависимостей.

Отчёт устроен по пунктам задания. Где что-то не сработало или не делалось — сказано прямо; это осознанно, а не забыто.

Резюме (полстраницы)

Собран рабочий платёжный HTTP-сервис: создать платёж, повторить с тем же ключом идемпотентности, получить статус, отменить. Идемпотентность и потокобезопасность — центральное требование контракта — реализованы и **проверены под нагрузкой** (100 параллельных запросов с одним ключом дают ровно один платёж).

Работа шла по повторяемому workflow: сначала спецификация контракта, затем **три субагента параллельно** написали три независимые реализации в трёх git worktree, каждая со своей моделью конкурентности. Объективный чёрно-ящичный conformance-набор (27 проверок по HTTP) прошёл на всех трёх — **25/25** на момент выбора. Выбрана самая пригодная к сопровождению (слоистая, impl-b), смержена в `main`. Затем **отдельный агент-ломатель** (не автор кода) 61 сценарием пытался сервис сломать: реальных багов не нашёл, но выявил 2 minor-несоответствия текста спеки — оба исправлены и покрыты тестами, conformance расширен до **27/27**.

Главный измеренный эффект параллельности — не столько время (хотя здесь и оно: 489 с против ~1175 с оценочно), сколько **контекст**: 225 К токенов работы с кодом остались в контекстах субагентов и не попали в главную сессию.

Обязательная часть

1. Работающий результат — выполнено

- Сервис в `service/`, запускается `go run .` (или `CGO_ENABLED=0 go run .`), инструкция — в `README.md`. Эндпоинты, коды ошибок и статусы — по `docs/SPEC.md`.
- Тесты: `go test -race ./...` зелёные (юнит + конкурентные); общий conformance — `python3 conformance/conformance.py service` → **27/27**.
- Хранилище in-memory (перезапуск обнуляет) — так и задумано для v1.0, отмечено в спеке.

2. Агенты — выполнено

- Три субагента запущены **параллельно, в одной сессии**, каждому выдан свой git worktree (`impl-a/b/c`) и своя «дизайн-полоса», чтобы реализации отличались:
- А — один `sync.Mutex` + `map` (простейшая корректность);
- В — слои `store / handlers / apperr`, `RWMutex` + резервирование ключа;
- С — `sync.Map` + атомарный `LoadOrStore`, хеш нормализованного тела.
- Четвёртый агент — **ломатель**, в отдельной сессии, не автор кода.
- Следы видны: ветки `impl-*` в истории, коммиты `merge`, журнал сессий с дословными промптами в `sessions/`, отчёты агентов в `reports/`.
- Замер параллельно/последовательно — `reports/measurement.md` (см. честную оговорку там).

3. Workflow — выполнено

- `workflows/spec-parallel-build.md` — процедура «спека → веер реализаций → ломатель → сведение», каждый шаг оставляет файл-артефакт. `workflows/PROMPT.txt` — готовый промпт для повторного запуска на любом контракте. Именно этой процедурой и собран весь проект.

PDF — выполнено

- Этот отчёт собран в `reports/report.pdf` (HTML + печатная CSS через headless Chrome: титул, оглавление, один акцентный цвет, таблицы, колонтитулы с номерами страниц).

Необязательная часть

git worktrees — выполнено

Три ветки создавались как реальные worktree (`git worktree add`), в каждой работал свой агент; лучшая (`impl-b`) смержена в `main` через `--no-ff`. Ветки `impl-a / impl-c` сохранены в истории как альтернативы. Worktree после сведения удалены (`git worktree remove + prune`) — за диском следил, как советовали на занятии.

Агент-ломатель — выполнено

61 сценарий, отчёт `reports/breaker.md`. Реальных багов 0; 2 minor-несоответствия исправлены. Отдельная сессия, не автор кода — как и требовалось.

Замер параллельно/последовательно — частично

Параллельный прогон измерен реально; полный последовательный — **не гонял** (стоил бы ещё ~20 мин и ~225 К токенов в главную сессию ради числа, выводимого из длительностей). Последовательная колонка в `reports/measurement.md` — честная оценка, а не факт.

Журнал сессий — выполнено

`sessions/` — дословные промпты всех агентов и ключевые решения.

Что НЕ делал (осознанно)

- **Изоляция с доказанным отказом** (песочница/контейнер) — не делал. Причина: go-сборка на этой машине и так конфликтует со встроенной песочницей Bash (см. ниже), а полноценный Docker-демонстратор выходил за рамки времени. Отдельного «вывода неудачной попытки» нет.
- **Инъекция на двух моделях** — не делал. Вариант C1 не про недоверенный текст; подкладывать инъекцию в платёжный JSON искусственно не стал.
- **Оптимизатор контекста / индекс кодовой базы** — не ставил. Проект маленький, эффект был бы незаметен; ставить сторонние хуки ради галочки не хотелось.

Что не сработало / трение (честно)

1. **Тулчейн Go на машине настроен под кросс-компиляцию.** `go env CC` указывал на `x86_64-w64-mingw32-gcc` (Windows), из-за чего `go test -race` (нужен `sgo`) не собирался с дефолтным компилятором. Все три агента-реализатора независимо на это наткнулись; один (B) поправил глобально `go env -w CC=gcc`. Сервис на чистой стандартной библиотеке, `sgo` ему не нужен — гоняется с `CGO_ENABLED=0`.
2. **Встроенная песочница Bash мешает go.** Спавнящийся из `go` компилятор не мог читать `/usr/include`; сборку/тесты приходилось запускать с отключённой песочницей. Ровно тот случай из материалов: песочница накрывает Bash и его дочерние процессы.
3. **PDF-конвейер пришлось выбирать под окружение.** `pip` / `venv` недоступны (нет `ensurepip`), `weasyprint` не поставить. Зато в системе есть Google Chrome — собрал PDF через него (`puppeteer-core` + системный Chrome, без скачивания браузера).
4. **Замер последовательного прогона не доведён до факта** — см. выше.

Ни один из пунктов не блокировал результат, но каждый стоил времени — и это ровно те «грабли окружения», ради которых задание и просит попробовать руками.

Ссылки на артефакты

Файл	Что
<code>service/</code>	реализация API (Go)
<code>docs/SPEC.md</code>	контракт v1.0
<code>conformance/conformance.py</code>	27 чёрно-ящичных проверок
<code>workflows/spec-parallel-build.md</code> , <code>PROMPT.txt</code>	workflow
<code>reports/comparison.md</code>	сравнение трёх реализаций
<code>reports/breaker.md</code>	отчёт агента-ломателя (61 сценарий)
<code>reports/measurement.md</code>	замер параллельно/последовательно
<code>sessions/</code>	журнал сессий с дословными промптами